

**KNOWLEDGE GRAPH EXTRACTION FROM
UNSTRUCTURED DOCUMENTS**

A PROJECT PHASE-2 REPORT

CHAPTER 6: REQUIREMENT SPECIFICATIONS

6.1 Feasibility Study (Technical, Economic, Operational)

A feasibility analysis was conducted across technical, economic, and operational dimensions prior to system design:

1. **Technical Feasibility:** Assesses the availability of open-source models, libraries, and frameworks. Utilizing Python 3.10+, PyTorch, Hugging Face Transformers, fastcoref, and Neo4j Community Edition provides a robust technical stack capable of executing complex NLP and graph storage operations locally or on standard cloud infrastructure.
2. **Economic Feasibility:** Evaluates the operational cost of deployment. By leveraging open-source foundation models (e.g., Llama-3-8B, REBEL, SpaCy) deployed on local GPU infrastructure (NVIDIA RTX GPUs or T4 instances), the system avoids heavy recurring per-token cloud API fees, ensuring long-term financial sustainability for enterprise deployment.
3. **Operational Feasibility:** Evaluates system adoption and workflow integration. The architecture features modular ingestion interfaces and automated Cypher query compilation, allowing domain analysts to query and explore Knowledge Graphs without needing deep manual Cypher scripting skills.

6.2 Functional Requirements

1. **Document Ingestion:** The system shall parse PDF, TXT, and HTML files, stripping irrelevant artifacts (headers, footers, page numbers).
2. **Text Segmentation & Coreference:** The pipeline must split text into semantically cohesive chunks while replacing pronouns with canonical referents.
3. **Entity and Relation Extraction:** The engine must identify entities (Person, Org, Location, Concept, etc.) and extract directed semantic relations between them.
4. **Graph Persistence:** The extracted structure must be converted to Cypher queries and ingested into a Neo4j Graph Database.
5. **Query Interface:** The system must support Cypher queries and Graph-based RAG natural language querying.

6.3 Non-Functional Requirements

6.3.1 **Accuracy & Precision:** Target relation extraction Precision > 80% on standard datasets.

6.3.2 **Scalability:** System architecture must support parallel, asynchronous chunk processing using task queues (e.g., Celery/Redis).

6.3.3 **Maintainability:** Modularity using microservices or isolated Python modules to allow swapping extraction models seamlessly.

6.3.4 **Extensibility:** The schema must support dynamic addition of novel entity types without requiring complete database rewrites.

6.4 Software Requirements

The software framework, runtime environments, and core dependencies required for system deployment are defined as follows:

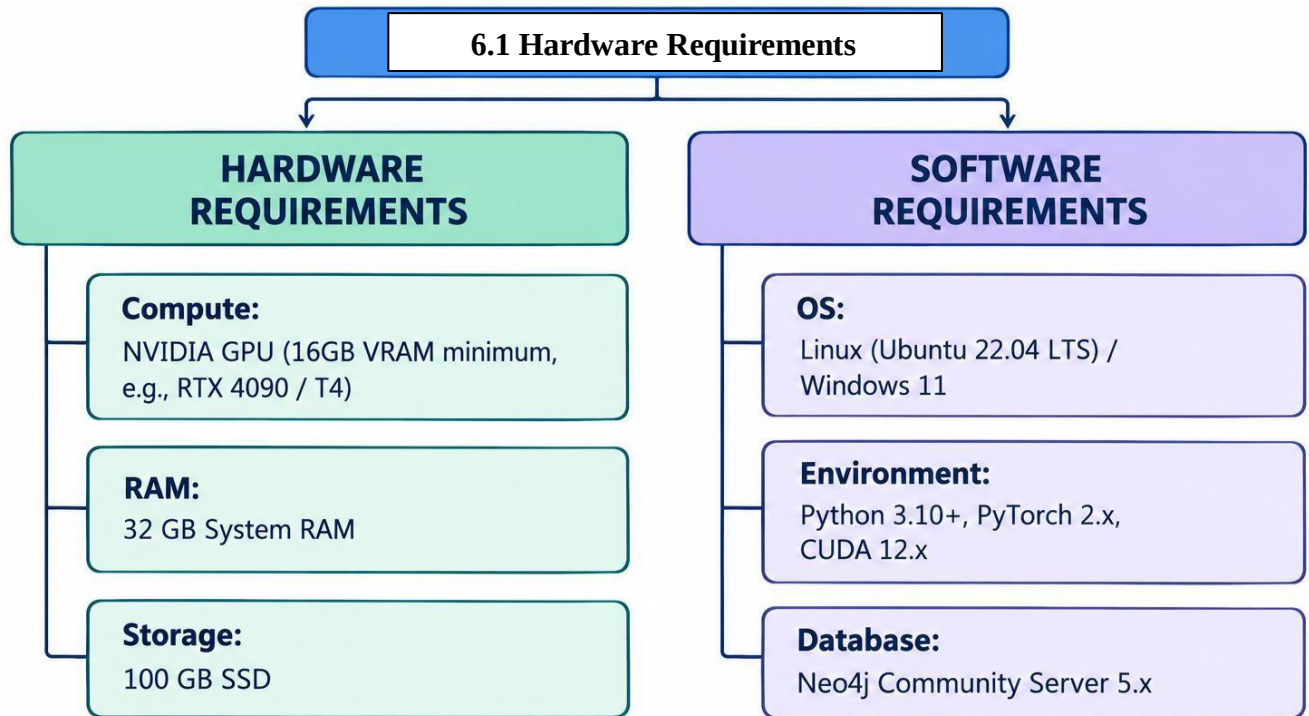
- **Operating System:** Linux Ubuntu 22.04 LTS or Windows 11 (64-bit).
- **Programming Language & Core Runtime:** Python 3.10+, PyTorch 2.x, CUDA 12.1 toolkit.
- **NLP & ML Libraries:** Hugging Face Transformers, SpaCy 3.7+, fastcoref, Sentence-Transformers, Pydantic v2.
- **Database Systems:** Neo4j Graph Database Enterprise / Community Server 5.x (Cypher engine), ChromaDB / FAISS (for vector similarity mapping).
- **Document Parsing Frameworks:** PyMuPDF (fitz), pdfplumber, python-docx, Unstructured.io.

6.5 Hardware Requirements

The recommended physical and compute infrastructure for executing end-to-end model training, fine-tuning, and graph persistence pipeline includes:

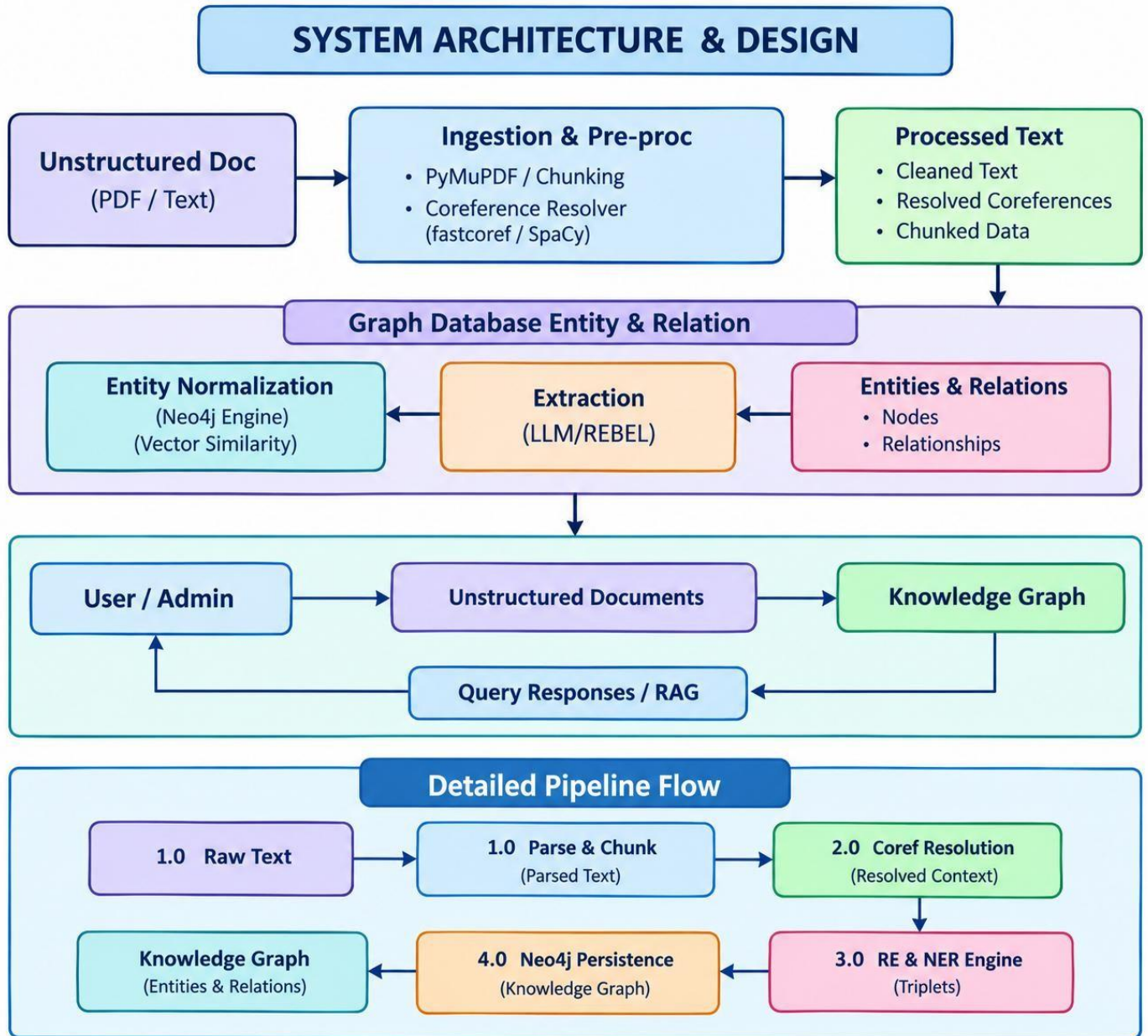
- **Compute / GPU:** NVIDIA GPU with a minimum of 16GB VRAM (e.g., RTX 4090 or NVIDIA T4/A10G) for efficient LLM inference and transformer embedding generation.
- **System Memory (RAM):** 32 GB System RAM for high-throughput batch text preprocessing and graph loading.

- **Storage:** 100 GB Solid State Drive (SSD) for model weight caching, dataset buffering, and Neo4j database files.



CHAPTER 7: SYSTEM ARCHITECTURE & DESIGN

7.1 Overall Pipeline Architecture



SYSTEM ARCHITECTURE & DESIGN

7.2 Data Flow Diagrams (DFD Level 0, Level 1, Level 2)

Data flow diagrams illustrate document ingestion at Level 0, text chunking and coreference resolution

at Level 1, and triple extraction alongside Cypher persistence in Neo4j at Level 2.

CHAPTER 8: MODULES DESCRIPTION

8.1 Document Ingestion & Cleansing Service

Raw unstructured text is extracted from documents using specialized parsers like PyMuPDF or pdfplumber. The extracted raw string is cleaned by normalizing whitespace, stripping non-printable Unicode characters, and separating multi-page documents into logically coherent chunks using a sliding window algorithm with an overlap factor:

$$\text{Window Size} = 512 \text{ tokens}, \quad \text{Overlap} = 64 \text{ tokens}$$

8.2 Coreference Resolution Module

Pronoun ambiguity represents a major source of missed relation extractions. The pipeline utilizes the fastcoref library (or LingMess model) to replace pronominal mentions with their explicit antecedents across multi-sentence sliding windows.

8.2.1 Original Text: *"Microsoft was founded by Bill Gates. He served as the CEO for many years."*

8.2.2 Resolved Text: *"Microsoft was founded by Bill Gates. Bill Gates served as the CEO for many years."*

8.3 Entity & Relation Extraction Engine

Entities and directed semantic relations are recognized using a dual-strategy joint extraction framework combining seq2seq translation and schema-constrained LLM prompting.

- 1. Transformer Encoder NER:** Fine-tuned dslim/bert-base-NER or spacy-en_core_web_trf to extract candidate entities tagged under standard types: PER (Person), ORG (Organization), LOC (Location), MISC (Miscellaneous).
- 2. Entity Canonicalization:** Extracted string names are mapped to normalized keys using String Similarity Metrics (Levenshtein Distance) and Vector Embedding Cosine Similarity to prevent duplicate node creations (e.g., merging *"IBM Corp."*, *"IBM"*, and *"International Business Machines"* into a single node key).

8.4 Entity Normalization & Canonicalization

Extracted string names are mapped to normalized keys using String Similarity Metrics (Levenshtein Distance) and Vector Embedding Cosine Similarity to prevent duplicate node creations (e.g., merging "IBM Corp.", "IBM", and "International Business Machines" into a single node key).

REBEL linearizes relation extraction into translation generation. The input text sentence is translated into a string structured with special control tokens (<triplet>, <subj>, <obj>): Input: "Steve Jobs co-founded Apple in Cupertino."

Output: <triplet> Steve Jobs <subj> Apple <obj> founded by <triplet> Apple <subj> Cupertino <obj> located in

8.5 Graph Storage & Persistence Layer (Neo4j)

Using Large Language Models (e.g., Llama-3-70B via vLLM or OpenAI GPT-4) coupled with strict Pydantic schemas to output structured triplets with explicit reasoning steps:

```
class KnowledgeTriplet(BaseModel):
```

```
    subject: str = Field(description="Normalized subject entity name")
```

```
    subject_type: str = Field(description="Category of subject entity")
```

```
    predicate: str = Field(description="Standardized relationship verb in UPPERCASE")
```

```
    object: str = Field(description="Normalized object entity name")
```

```
    object_type: str = Field(description="Category of object entity")
```

Extracted clean triplets are converted into Cypher statements executed using the official Neo4j Python Driver. `MERGE` clauses prevent node and relationship duplication while enforcing unique index constraints:

